Get a Pet — Backend, Frontend & Database Task Overview

2026-09-16 · @Someone

Status check ahead of tomorrow's team meeting: what each area has built, what's left to reach the MVP goal, and gaps worth flagging.

Project goal

MVP: a live webpage where users search European shelter and homeless dogs, filter results, view dog details, and get shelter or rescue organization/person contact info to start an adoption. Milestone 1 (per the roadmap) is a working prototype running on team members' machines, with backend, frontend, and database connected end to end.

Current state: backend and frontend have both been built out separately, but **are not yet connected** — that integration is the critical path to the milestone.

Done so far — Backend

Django 5.1 + Django REST Framework, with SQLite for local dev and a PostgreSQL switch available via `.env`.

- Models: `Shelter`, `Dog`, `ContactFormInquiry` — matching the vision doc's schema, simplified (no separate `Users table` yet)
- Full REST API via `DefaultRouter`: `/api/shelters/`, `/api/dogs/`, `/api/inquiries/`
- `Dog` list endpoint supports filtering by `country`, `sex`, `size`, `breed`, `adoption_status`
- Serializers include real validation (email format, non-empty message)
- Test suite covers filtering, dog detail, and inquiry submission/validation
- `seed_demo_data` management command with realistic fixtures (2 shelters, 3 dogs)
- Django admin registered for all three models
- Actively being worked on — the latest migration (adding `Shelter.country`) is from Sep 14, 2026

Done so far — Frontend

Built in three planned stages, progressing toward React/Next.js:

1. **HTML/CSS** — complete: Home (with search form), Search Results, one dog detail page (Mokka), Dog Upload/Shelter Login page. Real SEO metadata, JSON-LD structured data, semantic HTML, brand fonts and colors applied
2. **Tailwind CSS** — complete and more polished: same pages rebuilt larger with a real Tailwind v4 build pipeline (`input.css` → `output.css`), full brand color palette as a safelist
3. **JavaScript** — not started: placeholder files are empty

No React or Next.js code exists yet, despite that being the documented target framework.

All of this is currently static — the search results and dog pages use hardcoded fake data (Mokka, Mory, Lada, etc.), and most linked dog detail pages don't actually exist yet (only Mokka's page was built).

Key gaps and things to flag

Issue	Why it matters	Owner
No frontend↔backend connection at all	Nothing in the frontend calls `/api/` ; this is the whole point of issue #1	Frontend
No CORS configured on backend	Browser will block every frontend `fetch()` call until `django-cors-headers` is added	Backend
Filter mismatch: UI has `age` filter and multi-select `size` ; API only supports single-value `size` , no `age`	Silently dropping filled-in filters is a bad UX — needs a decision	Backend + Frontend
Most linked dog detail pages don't exist (only Mokka's)	Will 404 once real dog IDs are used unless the detail page is made generic	Frontend
No `MEDIA_URL` / `MEDIA_ROOT` configured	`Dog.photos` uploads won't be served correctly	Backend

Issue	Why it matters	Owner
No authentication implemented	Shelter Login UI exists but isn't wired to any real auth (JWT planned but not built)	Backend
Tiered pricing model in the upload flow (€9.90/dog, bulk discounts)	Wasn't in any planning doc; may conflict with the non-profit (MTÜ) framing discussed earlier — needs team alignment	Team decision
`requirements.txt` pins Django <6.0, but migration 0002 was generated by Django 6.1.1	Version drift between environments	Backend
Minor HTML bug: unclosed `<div>`s around the Shelter Login form	Quick fix	Frontend
No Dockerfile or CI/CD config yet	Expected eventually per the roadmap, not urgent for MVP	DevOps/later

Backend — remaining tasks

Now — MVP integration

- [ ] Install and configure `django-cors-headers` so the frontend can call the API from a different origin
- [ ] Decide with frontend: add `age` filtering and multi-value `size` filtering to `/api/dogs/`, or drop those fields from the UI
- [ ] Configure `MEDIA_URL` / `MEDIA_ROOT` so `Dog.photos` uploads are served correctly
- [ ] Reconcile the Django version pin (`requirements.txt` says <6.0, but a migration was generated with 6.1.1)
- [ ] Confirm the API contract documented in issue #1 is stable — frontend is building directly against it

Next — full product

- [ ] Authentication (JWT) for shelters: login, session handling, permission checks on write endpoints
- [ ] Shelter self-service: endpoints for a shelter to create/update/delete its own dog listings, scoped to their account

- [ ] Breed search across languages: research and implement matching breed keywords in dog descriptions across languages (raised repeatedly in Trello/Confluence, still unresolved)
- [ ] Multilingual backend support: integrate a translation API (Google Cloud Translation or DeepL) for dynamic content (dog descriptions), separate from frontend static-text translation
- [ ] Automations: profile-expiry notifications, adoption-status updates (mentioned in the roadmap, not started)
- [ ] Choose and configure production hosting (AWS vs DigitalOcean — still undecided) and set up deployment
- [ ] Add `a Users table` per the original schema
- [ ] Expand test coverage as new endpoints (auth, shelter dashboard) are added

Frontend — remaining tasks

Backend opened **issue #1** on Sep 14, assigning the MVP frontend-backend integration. Scope: dog search/listing, filtering, dog detail view, inquiry submission — explicitly excluding auth, shelter dashboard, search performance tuning, email notifications, and deployment.

Build plan (against `frontend/2-tailwind/`, the more current stage):

Now — MVP integration (issue #1)

- [ ] Create a small API service module (`5-js/api.js`) centralizing `fetchDogs()`, `fetchDogById()`, `submitInquiry()`
- [ ] Wire the search results page: read filters from the URL query string, call the API, render real dog cards, add loading/empty/error states
- [ ] Make the dog detail page generic: read `?id=` from the URL, fetch that dog, populate all fields dynamically instead of Mokka's hardcoded data
- [ ] Add a real adopter inquiry form (name, email, message) to the dog detail page, wired to `POST /api/inquiries/`, showing success and field-level validation errors
- [ ] Apply consistent loading/empty/error handling across both pages
- [ ] Fix the unclosed `<div>` bug on the Shelter Login form
- [ ] Test against every bullet in issue #1's acceptance criteria before calling it done

Next — full product

- [ ] Migrate from multi-page Tailwind HTML to React/Next.js, per the original framework decision — the JS stage (3) is currently empty and this hasn't started
- [ ] Build out the Shelter Dashboard (upload/manage listings) once backend self-service endpoints exist
- [ ] Wire the Shelter Login form to real JWT auth once backend supports it
- [ ] Implement frontend i18n (e.g. `react-i18next`) for static UI text; the language switcher in the header is currently non-functional
- [ ] Set up a frontend testing framework (Jest/Cypress/React Testing Library — all mentioned in the roadmap, none in place) and write tests for the search, detail, and inquiry flows
- [ ] Build remaining pages from the original design: Our Mission, Useful Info & Links
- [ ] Resolve the filter mismatch with backend (age, multi-select size) once a decision is made

Depends on backend adding CORS support first (see Backend section).

Database (PostgreSQL) — remaining tasks

The app currently runs on SQLite locally; PostgreSQL is wired up via `.env` but not confirmed running against the real schema.

Now — MVP

- [ ] Stand up PostgreSQL for local dev per the wiki's install guide, and switch `.env` to the Postgres settings
- [ ] Verify the current Django models cover the wiki's fuller schema (e.g., `a Users table is` n't in Django yet) and reconcile differences
- [ ] Confirm indexing plan for the filtered fields (`country` , `sex` , `size` , `breed` , `adoption_status`) as data volume grows

Next — full product

- [ ] Apply the documented naming conventions (lowercase snake_case, `_v` for views, `pk_` / `fk_` / `idx_` prefixes, `live` schema) to the Django-managed tables, or confirm the team is consciously not following them under Django's ORM defaults
- [ ] Create the documented accounts/roles (`admin` , `developer` , `application` , `analyst` groups; personal accounts like `pkolk`) on the real Postgres instance, matching the wiki's scripts

- [ ] Decide on and document the audit fields (`date_created`, `date_updated`, `created_by`, `updated_by`) — not yet present on the Django models
- [ ] Plan for scale: partitioning strategy for the `dogs` table (by species/country) and an archiving approach for adopted animals, per the original vision doc
- [ ] Set up production database hosting (managed Postgres on AWS RDS or DigitalOcean) once a hosting decision is made

Open questions for tomorrow

- [ ] Age filter and multi-select size: extend the API, or trim the UI to match what's supported?
- [ ] Is the tiered dog-upload pricing confirmed as a real decision — needs the rate structure finalized and reconciling with the non-profit/MTÜ framing.
- [ ] Who owns standing up PostgreSQL for local dev vs. staying on SQLite through the MVP demo?
- [ ] Target date for issue #1 (frontend-backend integration) — is it this sprint?
- [ ] Should JWT auth for shelters be scoped now or stay explicitly out of MVP as issue #1 states?
- [ ] The frontend logo and nav now say "Dogeli" instead of "Dogs of Europe" — confirm this is an intentional team decision, and if so, update domains, legal registration name, and remaining branding (favicon, meta tags, structured data) to match

Non-technical work

Legal & business structure

- [ ] Finalize the hybrid MTÜ (non-profit mission) + OÜ (commercial arm) structure researched earlier
- [ ] Register the brand name in Estonia (~~€20) and, later, across the EU (~~€800)
- [ ] Reconcile the tiered dog-upload pricing model with the non-profit framing (confirmed as a real decision — see Open questions above)

Compliance

- [ ] Turn the GDPR/data-privacy research (discussed at length in Trello) into concrete actions: privacy policy, consent flows, data retention rules for user/shelter data

Content & shelter onboarding

- [ ] Contact and onboard real shelters and rescue organizations to populate the platform with genuine dog listings
- [ ] Build the Excel/shared-sheet of European shelters (in progress per Trello) into an actual onboarding pipeline
- [ ] Define the process for verifying a shelter before they can list dogs

Project management

- [ ] Attach target dates to the technical tasks above — none currently have deadlines
- [ ] Remove Aleksander's and Khalid's access to Confluence wiki and Trello Kanban, and review whether their GitHub access should be revoked too